

SENA — ADSO 2026

Clase 33

Diseño Técnico del Proyecto

API de Eficiencia de Operarios

Texticode — Sistema de Gestión Textil

Aprendices

Kevin Stevan Wagner
Kevin Andres Castro
Andres Felipe Lancheros
Camilo Andres Tibambre

Instructor

Ing. Edwin Marín

Fecha

Mayo 2026

Programa

ADSO 2026

1. Arquitectura 3 Capas

Texticode implementa una arquitectura REST desacoplada en 3 capas. El frontend nunca accede directamente a la base de datos — toda comunicación ocurre vía HTTP con autenticación por API Key.

FRONTEND Vue.js 3 · EficienciaOperarios.vue · Axios HTTP · VITE_API_KEY en .env
↓ HTTP / REST (x-api-key)
BACKEND Node.js + Express · routes/eficiencia.js · middleware apiKey.js · Lógica de negocio
↓ SQL queries (pg.Pool — Supabase Session Pooler 6543)
BASE DE DATOS Supabase PostgreSQL · tablas: usuario, rol, orden_produccion, observacion_operario

FLUJO DE DATOS — CONSULTAR EFICIENCIA DE OPERARIOS

Capa	Acción
Frontend	EficienciaOperarios.vue → axios.get('/api/eficiencia/operarios', { headers: { 'x-api-key': KEY } })
Backend Auth	middleware apiKey.js verifica x-api-key contra API_KEY_EFICIENCIA en .env
Backend Business	eficiencia.js calcula prendas/día ponderadas (Alta=3, Media=2, Baja=1)
Persistence	pg.Pool ejecuta SELECT + JOIN en Supabase PostgreSQL (Session Pooler 6543)
Database	PostgreSQL devuelve registros con garantías ACID
Response	200 OK + { ok: true, total, filtros_aplicados, data: [...] }

2. Contrato Técnico API

Los 4 contratos de la API de Eficiencia documentados con evidencia real de pruebas en Thunder Client.

2.1 GET /api/eficiencia/operarios

GET /api/eficiencia/operarios

Lista todos los operarios activos con métricas de rendimiento calculadas en tiempo real.

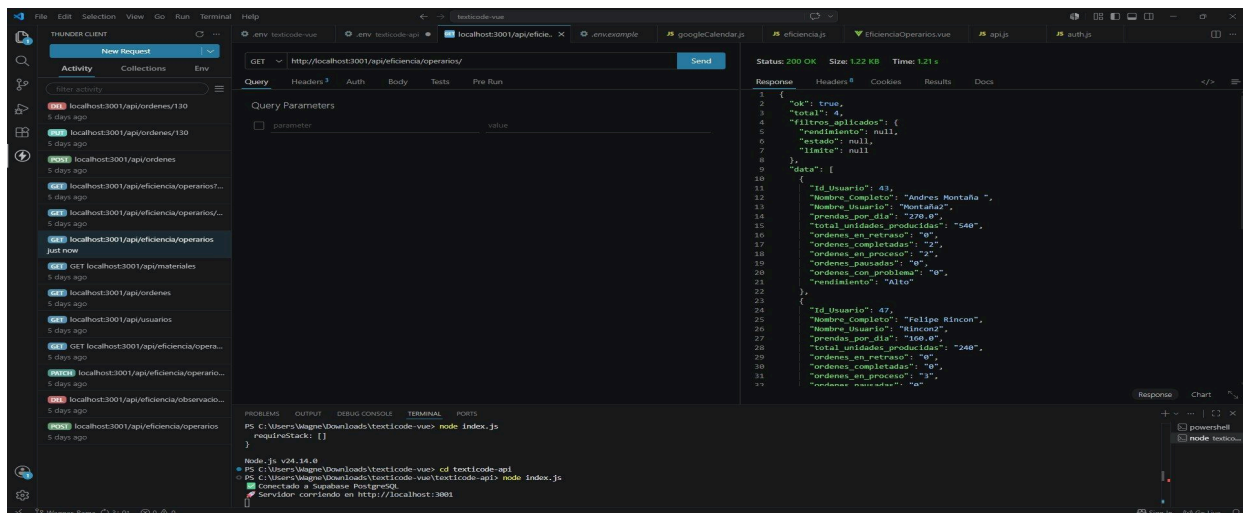
REQUEST

```
GET /api/eficiencia/operarios HTTP/1.1
Host: localhost:3001
x-api-key: texticode-2026

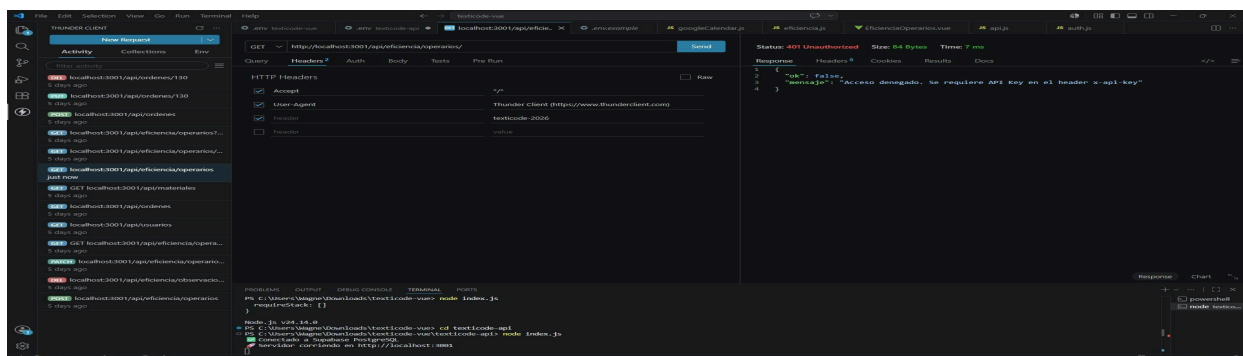
// Query params opcionales
?rendimiento=Alto|Medio|Bajo
?estado=Completada|En Proceso|Pausado
?limite=5
```

EVIDENCIA THUNDER CLIENT

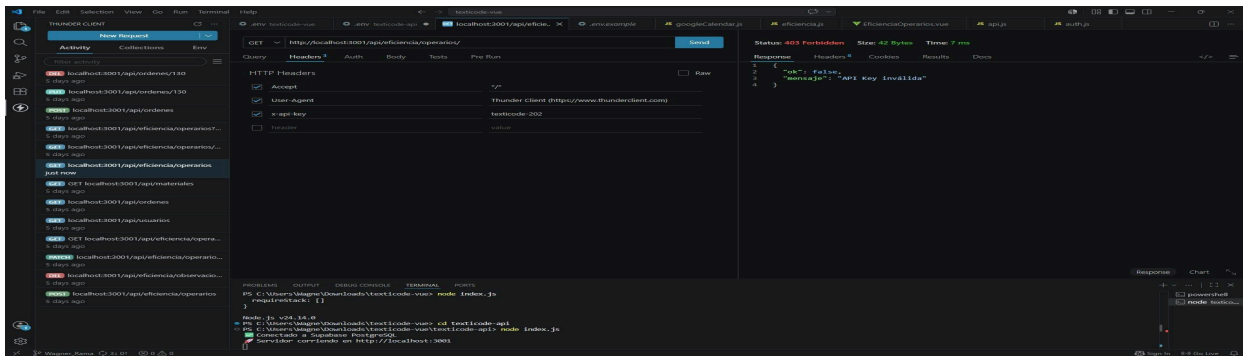
Prueba 1 — API Key correcta → 200 OK



Prueba 2 — Sin header x-api-key → 401 Unauthorized



Prueba 3 — API Key incorrecta → 403 Forbidden



2.2 GET /api/eficiencia/operarios/:id

GET /api/eficiencia/operarios/:id

Devuelve perfil completo de un operario con órdenes, métricas y observaciones.

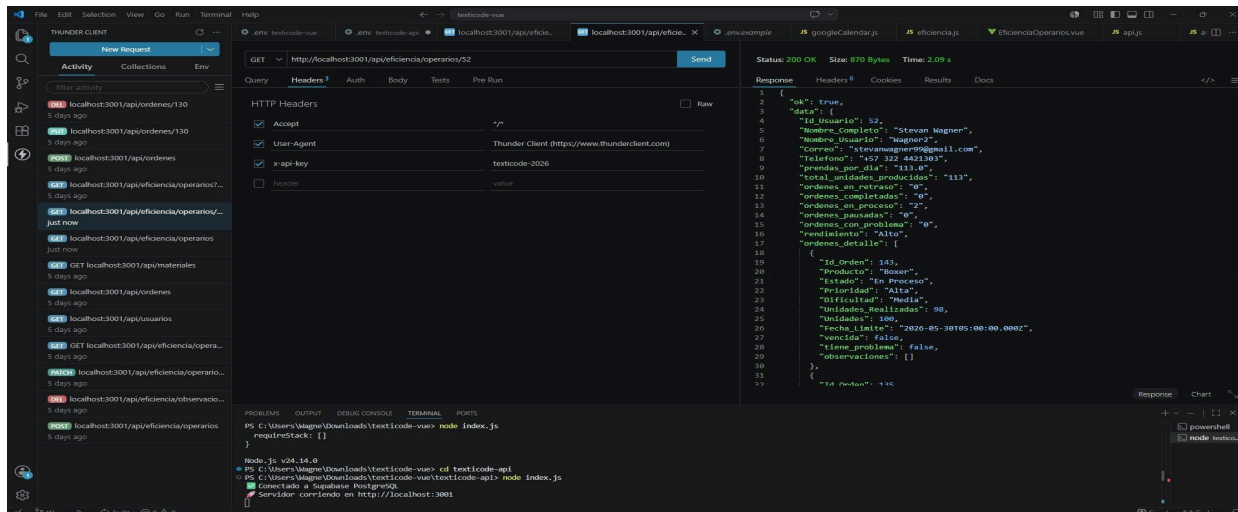
REQUEST

```
GET /api/eficiencia/operarios/52 HTTP/1.1
Host: localhost:3001
x-api-key: texticode-2026

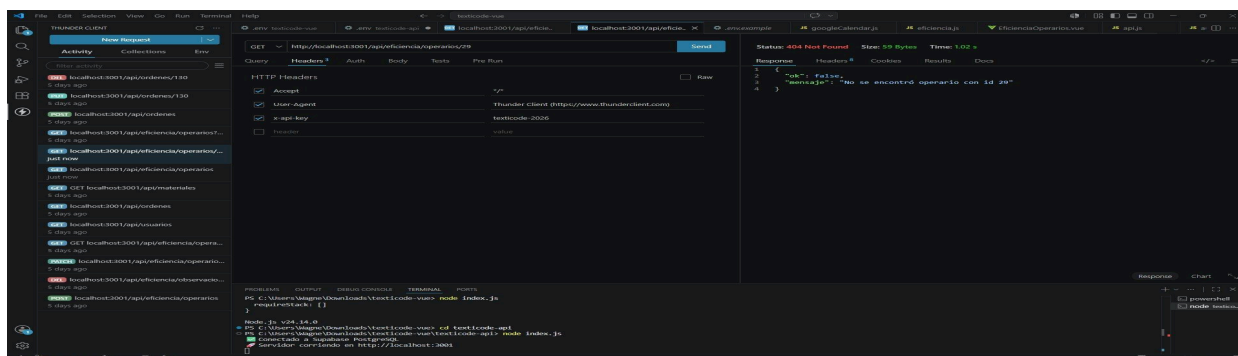
// :id → Id_Usuario del operario (entero positivo)
```

EVIDENCIA THUNDER CLIENT

Prueba 1 — Operario existente (id: 52 — Stevan Wagner) → 200 OK



Prueba 2 — Operario inexistente (id: 29) → 404 Not Found



2.3 POST /api/eficiencia/observaciones

POST /api/eficiencia/observaciones

Registra una nueva observación sobre el rendimiento de un operario en una orden específica.

REQUEST

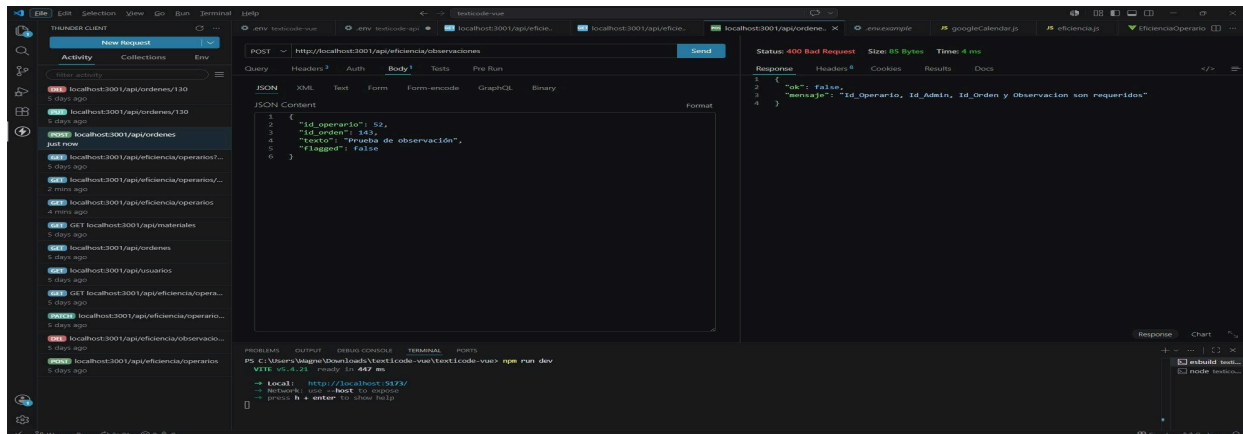
```
POST /api/eficiencia/observaciones HTTP/1.1
Host: localhost:3001
x-api-key: texticode-2026
Content-Type: application/json
```

```
{
  "Id_Operario": 52,
  "Id_Admin": 43,
  "Id_Orden": 143,
  "Observacion": "Prueba de observación"
}

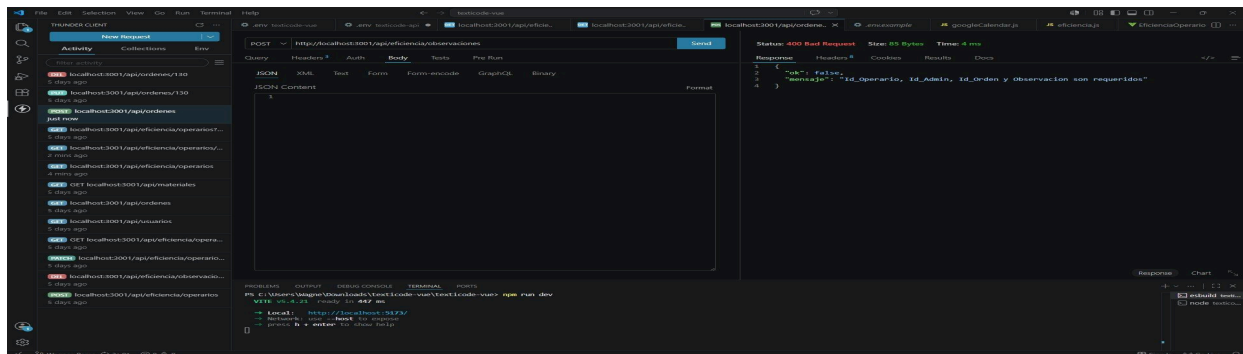
// Todos los campos son requeridos. Observacion máx. 500 caracteres.
```

EVIDENCIA THUNDER CLIENT

Prueba 1 — Body completo y correcto → 201 Created



Prueba 2 — Body vacío (campos faltantes) → 400 Bad Request



2.4 DELETE /api/eficiencia/observaciones/:id

DELETE /api/eficiencia/observaciones/:id

Elimina una observación de eficiencia por su Id_Observacion.

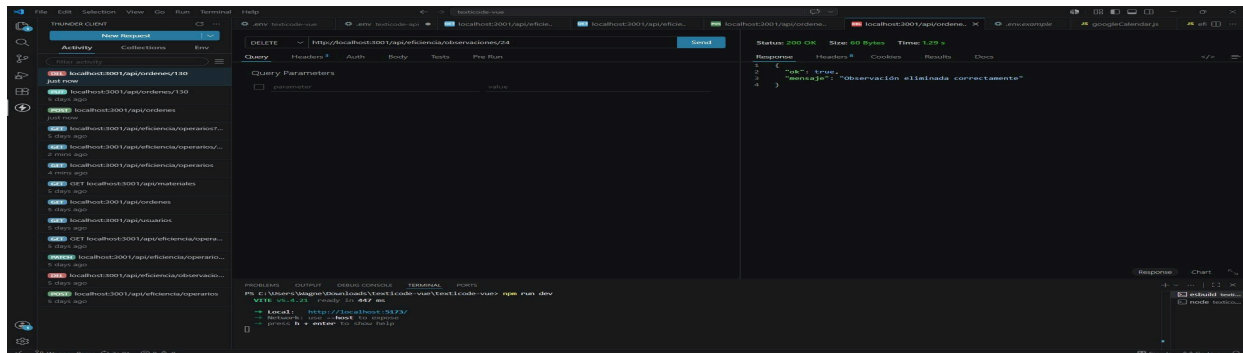
REQUEST

```
DELETE /api/eficiencia/observaciones/24 HTTP/1.1
Host: localhost:3001
x-api-key: texticode-2026

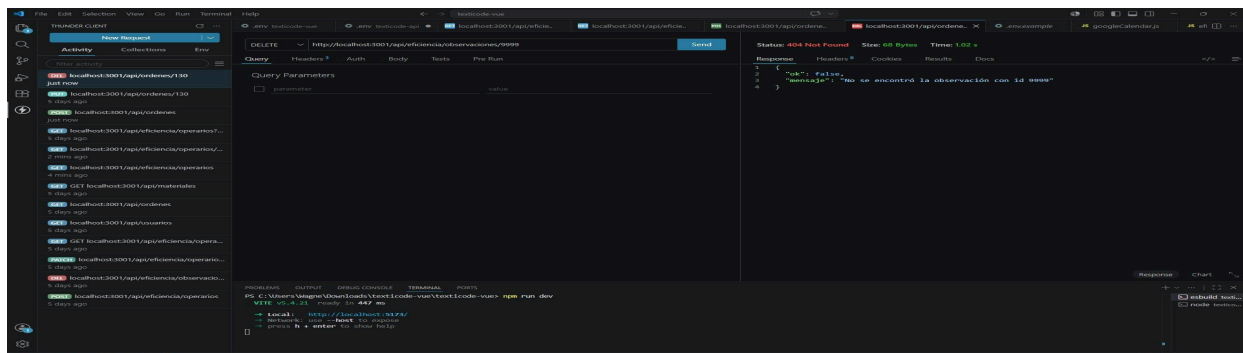
// :id → Id_Observacion (entero positivo)
```

EVIDENCIA THUNDER CLIENT

Prueba 1 — Observación existente (id: 24) → 200 OK



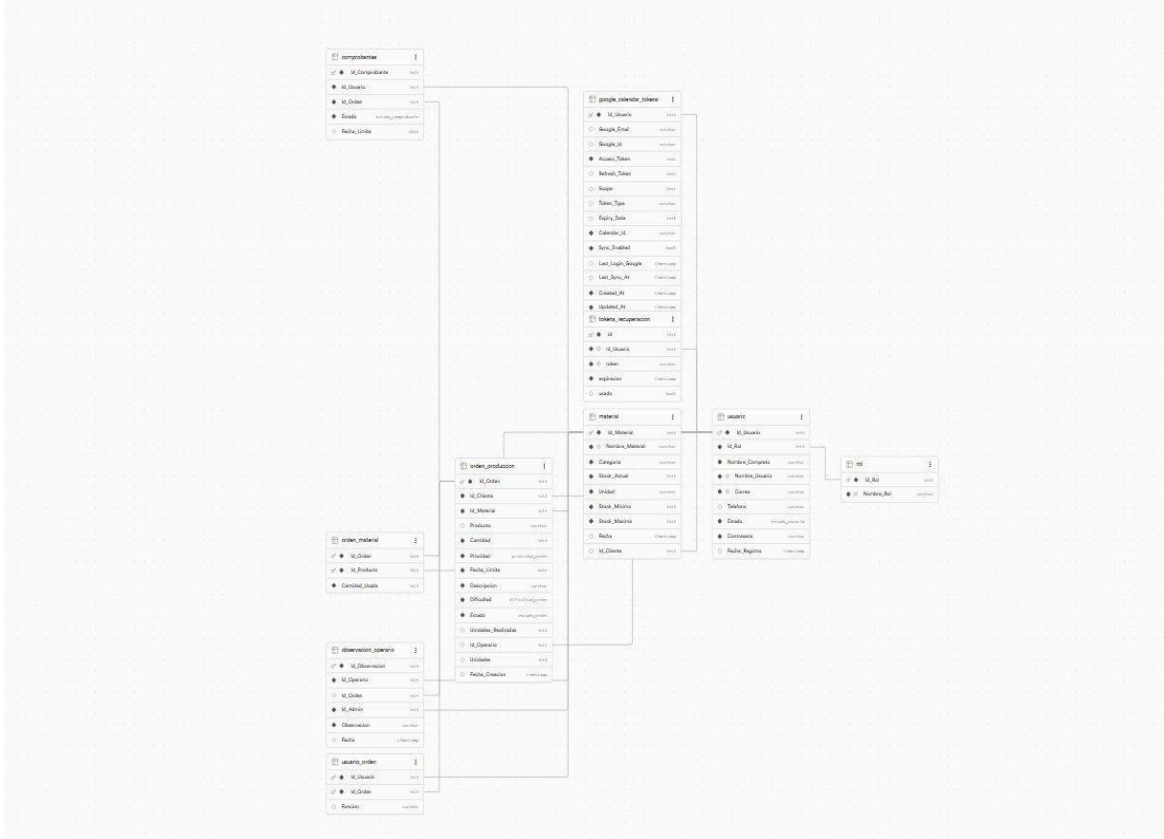
Prueba 2 — Observación inexistente (id: 9999) → 404 Not Found



3. Diagrama Entidad-Relación

El módulo de eficiencia opera sobre 4 tablas en Supabase PostgreSQL. La tabla central es observacion_operario, que vincula operarios, órdenes y administradores.

Tabla	Campos
usuario	Id_Usuario (PK) · Nombre_Completo · Nombre_Usuario · Correo (UNIQUE) · Telefono · Estado · Id_Rol (FK)
rol	Id_Rol (PK) · Nombre_Rol (admin operario cliente)
orden_produccion	Id_Orden (PK) · Id_Operario (FK) · Id_Cliente (FK) · Id_Material (FK) · Producto · Cantidad · Prioridad · Dificultad · Estado · Unidades · Unidades_Realizadas · Fecha_Limite · Fecha_Creacion
observacion_operario	Id_Observacion (PK) · Id_Operario (FK) · Id_Orden (FK) · Id_Admin (FK) · Observacion · Fecha
material	Id_Material (PK) · Nombre_Material · Categoria · Stock_Actual · Stock_Minimo · Stock_Maximo · Unidad · Fecha · Id_Cliente (FK)
orden_material	Id_Orden (FK) · Id_Producto (FK) · Cantidad_Usada



RELACIONES CLAVE

Origen	Relación	Destino	Descripción
usuario	1 → N	orden_produccion	Un operario tiene múltiples órdenes asignadas
usuario	1 → N	observacion_operario	Un operario puede acumular múltiples observaciones
orden_produccion	1 → N	observacion_operario	Una orden puede tener múltiples observaciones
rol	1 → N	usuario	Un rol agrupa múltiples usuarios
material	1 → N	orden_produccion	Un material puede usarse en múltiples órdenes
orden_produccion	1 → N	orden_material	Una orden registra el uso de materiales

4. Validaciones y Manejo de Errores

Todos los endpoints comparten el middleware apiKey.js y siguen una política uniforme de respuesta de errores.

Escenario	Validación en código	HTTP
API Key ausente	<code>!req.headers['x-api-key']</code>	401 Unauthorized
API Key incorrecta	<code>key !== process.env.API_KEY_EFICIENCIA</code>	403 Forbidden
ID no numérico	<code>isNaN(id)</code>	400 Bad Request
Operario no encontrado	<code>rows.length === 0</code>	404 Not Found
Observación no encontrada	<code>obs.length === 0</code>	404 Not Found
Campos requeridos faltantes	<code>!Id_Operario !Id_Admin !Id_Orden !Observacion</code>	400 Bad Request
Observacion > 500 chars	<code>Observacion.length > 500</code>	400 Bad Request
Orden no pertenece al operario	<code>SELECT orden WHERE Id_Operario = ? → rowCount 0</code>	400 Bad Request
Error interno de BD	<code>catch(err) → log interno</code>	500 Server Error

MIDDLEWARE APIKEY.JS

```
const verificarApiKey = (req, res, next) => {
  const key = req.headers['x-api-key']
  if (!key)
    return res.status(401).json({ ok: false,
      mensaje: 'Acceso denegado. Se requiere API Key en el header x-api-key' })
  if (key !== process.env.API_KEY_EFICIENCIA)
    return res.status(403).json({ ok: false, mensaje: 'API Key inválida' })
  next()
}
export default verificarApiKey
```